

Artificial Intelligence and Data Engineering for Orbital and Space Observation Systems

Laboratory 10: Explainable AI for Earth Observation

Leszek Siwik
Faculty of Space Technologies
AGH University of Krakow

1 Introduction

In previous laboratories, you built increasingly more advanced image classification pipelines for Earth Observation systems.

You started with:

- classical machine learning methods,
- convolutional neural networks,
- transfer learning,
- fine-tuning pretrained models.

Until now, the main focus was:

Can the model correctly classify satellite images?

However, in operational systems this is usually not sufficient.

A prediction alone does not explain:

- why the model made a decision,
- which image regions influenced the prediction,
- whether the model focused on meaningful structures,
- whether the decision can be trusted.

For example, suppose a model predicts:

<code>Predicted class: river</code> <code>Confidence: 0.94</code>
--

Several questions immediately appear:

- Did the model actually look at the river?
- Did it focus on surrounding vegetation?
- Did it learn some irrelevant visual artifact?

- Can we trust this prediction?

In this laboratory, you will introduce:

Explainable Artificial Intelligence (XAI)

Explainable AI refers to methods that help understand and interpret the behavior of machine learning models.

Instead of asking only:

What prediction did the model produce?

XAI also asks:

Why was this prediction made?

and

Which parts of the input data influenced the decision?

In this laboratory, you will investigate several visual explanation techniques for convolutional neural networks, including:

- Grad-CAM,
- HiResCAM,
- LayerCAM,
- EigenCAM,
- Occlusion Sensitivity Analysis.

These methods belong to different categories of explainability approaches.

CAM-based methods use internal feature maps and gradients to estimate attention regions.

Occlusion-based methods investigate how predictions change when parts of an image are removed.

For example:

```
input image  
prediction  
explanation method  
attention map  
interpretation
```

The goal is not only to visualize model attention but also to understand:

- whether the model focuses on meaningful structures,
- whether different methods produce similar explanations,
- whether suspicious behavior appears,
- whether the prediction can be trusted.

2 Why Explainability Matters

High accuracy alone is often insufficient.

In Earth Observation and space systems, AI models may support:

- autonomous orbital operations,
- environmental monitoring,
- mission planning,
- hazard detection,
- onboard decision systems.

Incorrect predictions may create risks such as:

- false land-cover interpretation,
- missed anomalies,
- incorrect mission decisions,
- unreliable autonomous behavior.

Therefore, engineers often ask:

Can we understand why the model produced a particular output?

Explainability does not guarantee correctness.

Instead, it helps identify:

- suspicious predictions,
- unexpected model behavior,
- dataset bias,
- incorrect learned patterns.

3 Project Structure

The project structure becomes:

```
data/  
  
models/  
  
reports/  
  gradcam_examples/  
  gradcam_analysis.txt  
  
src/  
  vision/  
    create_gradcam.py
```

Create required folders:

```
mkdir -p reports/gradcam_examples
```

4 Task 1: Reuse Trained Transfer Learning Model

This laboratory reuses the model trained during Laboratory 9.

Use:

```
models/resnet18_transfer.pt
```

or:

```
models/resnet18_finetuned.pt
```

For simplicity, begin with:

```
models/resnet18_transfer.pt
```

5 Task 2: What is Grad-CAM?

CNN models process images through multiple layers.

Earlier layers typically learn:

- edges,
- color transitions,
- simple textures.

Deeper layers often learn:

- shapes,
- larger structures,
- task-specific visual patterns.

Grad-CAM uses:

- feature maps from deep convolutional layers,
- gradients computed during backpropagation.

The general process is:

```
input image  
forward pass  
prediction score  
backward pass  
gradients  
weighted feature maps  
heatmap
```

The heatmap shows:

Which image regions contributed most strongly to the prediction.

6 Task 3: Install Additional Libraries

Install:

```
python3 -m pip install grad-cam opencv-python
```

7 Task 4: Create Grad-CAM Script

Create:

```
src/vision/create_gradcam.py
```

Add imports:

```
from pathlib import Path

import torch
from torch import nn
from torchvision import models, transforms

from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

from pytorch_grad_cam import GradCAM
from pytorch_grad_cam.utils.image import show_cam_on_image
```

8 Task 5: Load Trained Model

Add:

```
MODEL_PATH = Path(
    "models/resnet18_transfer.pt"
)

CLASS_NAMES_PATH = Path(
    "models/resnet18_classes.txt"
)
```

Add:

```
def load_class_names():

    with open(CLASS_NAMES_PATH) as f:
        class_names = [
            line.strip()
            for line in f
        ]

    return class_names
```

Add:

```
def load_model(class_names):

    model = models.resnet18(
```

```

        weights=None
    )

    input_features = model.fc.in_features

    model.fc = nn.Linear(
        input_features,
        len(class_names)
    )

    state_dict = torch.load(
        MODEL_PATH,
        map_location="cpu"
    )

    model.load_state_dict(
        state_dict
    )

    model.eval()

    return model

```

9 Task 6: Load Image

Add:

```

def load_image(image_path):

    image = Image.open(
        image_path
    ).convert("RGB")

    transform = transforms.Compose([
        transforms.Resize((224,224)),
        transforms.ToTensor()
    ])

    tensor = transform(
        image
    )

    return image, tensor.unsqueeze(0)

```

10 Task 7: Generate Prediction

Add:

```

def predict(
    model,
    image_tensor,
    class_names
):

    with torch.no_grad():

```

```

        outputs = model(
            image_tensor
        )

        probabilities = torch.softmax(
            outputs,
            dim=1
        )

        confidence, predicted = torch.max(
            probabilities,
            dim=1
        )

        predicted_class = class_names[
            predicted.item()
        ]

        print(
            f"Prediction: {predicted_class}"
        )

        print(
            f"Confidence: {confidence.item():.4f}"
        )

        return predicted.item()

```

11 Task 8: Generate Grad-CAM Heatmap

For ResNet18, use the last convolutional block:

```
model.layer4[-1]
```

Add:

```

def create_heatmap(
    model,
    image_tensor
):
    target_layers = [
        model.layer4[-1]
    ]

    cam = GradCAM(
        model=model,
        target_layers=target_layers
    )

    grayscale_cam = cam(
        input_tensor=image_tensor
    )

    return grayscale_cam[0]

```

12 Task 9: Visualize Results

Add:

```
def visualize(
    image,
    heatmap
):

    image = image.resize(
        (224,224)
    )

    image_array = (
        np.array(image)
        .astype(np.float32)
        /255
    )

    visualization = show_cam_on_image(
        image_array,
        heatmap,
        use_rgb=True
    )

    plt.figure(
        figsize=(10,5)
    )

    plt.subplot(1,2,1)
    plt.imshow(image)
    plt.title(
        "Original Image"
    )
    plt.axis("off")

    plt.subplot(1,2,2)
    plt.imshow(
        visualization
    )

    plt.title(
        "Grad-CAM"
    )

    plt.axis("off")

    plt.tight_layout()

    plt.savefig(
        "reports/gradcam_examples/example.png"
    )

    plt.show()
```

13 Task 10: Main Function

Add:


```
def main():

    class_names = (
        load_class_names()
    )

    model = load_model(
        class_names
    )

    image_path = (
        "data/processed/images/"
        "test/river/river_0000.jpg"
    )

    image, tensor = (
        load_image(
            image_path
        )
    )

    predict(
        model,
        tensor,
        class_names
    )

    heatmap = (
        create_heatmap(
            model,
            tensor
        )
    )

    visualize(
        image,
        heatmap
    )

if __name__=="__main__":
    main()
```

Run:

```
python3 -m src.vision.create_gradcam
```

14 Expected Result

You should observe:

- original satellite image,
- Grad-CAM heatmap,
- image with attention overlay.

Red regions indicate stronger influence on the prediction.
Blue regions indicate weaker influence.

15 Task 11: Analyze Model Attention

Test:

- forest images,
- river images,
- residential images,
- unknown images,
- noise images.

Save observations:

```
reports/gradcam_analysis.txt
```

Use:

```
GRAD-CAM ANALYSIS
=====

Prediction:
Confidence:

Observed attention:

Interpretation:

Conclusions:
- Did the model focus on meaningful regions?
- Did suspicious predictions appear?
- Can explainability guarantee correctness?
- Why is XAI important in operational systems?
```

16 Additional Experiments: Interpreting Model Behavior

So far, you tested Grad-CAM on a single correctly classified image.

Now perform additional experiments and analyze how the model behaves under different situations.

The goal is not only to obtain predictions but also to investigate:

- where the model focuses its attention,
- whether attention regions appear meaningful,
- how explanations change across classes,
- how the model behaves on unknown inputs.

Experiment 1: Forest Image

Modify:

```
image_path
```

inside:

```
main()
```

Replace:

```
image_path = (  
"data/processed/images/"  
"test/river/river_0000.jpg"  
)
```

with:

```
image_path = (  
"data/processed/images/"  
"test/forest/forest_0000.jpg"  
)
```

Run:

```
python3 -m src.vision.create_gradcam
```

Observe:

- predicted class,
- confidence,
- attention regions.

Questions:

- Does the model focus on vegetation regions?
- Does it look at textures or boundaries?
- Does attention appear reasonable?

Experiment 2: Residential Image

Modify:

```
image_path = (  
"data/processed/images/"  
"test/residential/residential_0000.jpg"  
)
```

Run:

```
python3 -m src.vision.create_gradcam
```

Questions:

- Does the model focus on building patterns?
- Does it focus on regular geometric structures?
- Are roads or buildings emphasized?

Experiment 3: Unknown Input (Highway)

Until now, the model was trained only on:

```
forest
residential
river
```

However, EuroSAT contains additional classes.

Use an image that was not part of training.

Replace:

```
image_path
```

with:

```
image_path = (
    "data/raw/eurosat/2750/"
    "Highway/Highway_1.jpg"
)
```

Run:

```
python3 -m src.vision.create_gradcam
```

Questions:

- Which class did the model predict?
- Was confidence high or low?
- Did Grad-CAM focus on meaningful regions?
- Did the model force a known class prediction?

Experiment 4: Random Noise Image

Create a random image.

Create:

```
src/data/create_noise_image.py
```

Add:

```
import numpy as np
from PIL import Image
from pathlib import Path
```

```

OUTPUT_PATH = Path(
    "data/inference_samples/noise.jpg"
)

OUTPUT_PATH.parent.mkdir(
    parents=True,
    exist_ok=True
)

noise = np.random.randint(
    0,
    255,
    (224,224,3),
    dtype=np.uint8
)

image = Image.fromarray(
    noise
)

image.save(
    OUTPUT_PATH
)

print(
    f"Saved: {OUTPUT_PATH}"
)

```

Run:

```
python3 -m src.data.create_noise_image
```

Now replace:

```
image_path
```

with:

```

image_path = (
    "data/inference_samples/noise.jpg"
)

```

Run:

```
python3 -m src.vision.create_gradcam
```

Questions:

- Did the model still produce a prediction?
- Was confidence high?
- Did Grad-CAM focus on specific regions?
- Did attention appear meaningful?

Final Analysis

Save conclusions in:

```
reports/gradcam_analysis.txt
```

Use the following structure:

```
GRAD-CAM ANALYSIS
=====

Experiment 1: Forest
Prediction:
Confidence:
Observed attention:
Interpretation:

Experiment 2: Residential
Prediction:
Confidence:
Observed attention:
Interpretation:

Experiment 3: Highway
Prediction:
Confidence:
Observed attention:
Interpretation:

Experiment 4: Noise
Prediction:
Confidence:
Observed attention:
Interpretation:

Final Conclusions:

- Did the model focus on meaningful regions?
- Did Grad-CAM help explain predictions?
- Did unknown inputs produce suspicious behavior?
- Can explainability guarantee correctness?
- Why is XAI important in operational Earth Observation systems?
```

17 Comparing Different CAM Methods

So far, you used Grad-CAM as the main visual explanation method.

However, Grad-CAM is not the only class activation mapping method.

The library used in this laboratory also provides several related methods, such as:

- Grad-CAM,
- HiResCAM,
- EigenCAM,
- LayerCAM.

Different XAI methods may produce different visual explanations.

This is important because:

There is no single universal explanation method that is always correct.

In this optional extension, you will compare several CAM methods on the same satellite image.

Task: Extend Imports

In:

```
src/vision/create_gradcam.py
```

replace:

```
from pytorch_grad_cam import GradCAM
```

with:

```
from pytorch_grad_cam import GradCAM, HiResCAM, EigenCAM, LayerCAM
```

Task: Create Multi-CAM Function

Add the following function:

```
def create_cam_by_method(
    model,
    image_tensor,
    method_name
):
    target_layers = [
        model.layer4[-1]
    ]

    methods = {
        "GradCAM": GradCAM,
        "HiResCAM": HiResCAM,
        "EigenCAM": EigenCAM,
        "LayerCAM": LayerCAM
    }

    cam_class = methods[
        method_name
    ]

    cam = cam_class(
        model=model,
        target_layers=target_layers
    )

    grayscale_cam = cam(
        input_tensor=image_tensor
    )

    return grayscale_cam[0]
```

Task: Visualize Multiple CAM Methods

Add:

```
def visualize_multiple_cams(
    image,
    heatmaps
):

    image = image.resize(
        (224,224)
    )

    image_array = (
        np.array(image)
        .astype(np.float32)
        /255.0
    )

    plt.figure(
        figsize=(16,8)
    )

    plt.subplot(2,3,1)
    plt.imshow(image)
    plt.title("Original")
    plt.axis("off")

    index = 2

    for method_name, heatmap in heatmaps.items():

        visualization = show_cam_on_image(
            image_array,
            heatmap,
            use_rgb=True
        )

        plt.subplot(
            2,
            3,
            index
        )

        plt.imshow(
            visualization
        )

        plt.title(
            method_name
        )

        plt.axis(
            "off"
        )

        index += 1

    plt.tight_layout()
```



```

output_path = (
    "reports/gradcam_examples/"
    "cam_methods_comparison.png"
)

plt.savefig(
    output_path
)

plt.show()

print(
    f"Saved: {output_path}"
)

```

Task: Add Main Function for CAM Comparison

Create a new file:

```
src/vision/compare_cam_methods.py
```

Add:

```

from src.vision.create_gradcam import (
    load_class_names,
    load_model,
    load_image,
    predict,
    create_cam_by_method,
    visualize_multiple_cams
)

def main():

    class_names = load_class_names()

    model = load_model(
        class_names
    )

    image_path = (
        "data/processed/images/"
        "test/river/river_0000.jpg"
    )

    image, tensor = load_image(
        image_path
    )

    predict(
        model,
        tensor,
        class_names
    )

```

```

methods = [
    "GradCAM",
    "HiResCAM",
    "EigenCAM",
    "LayerCAM"
]

heatmaps = {}

for method_name in methods:

    heatmaps[
        method_name
    ] = create_cam_by_method(
        model,
        tensor,
        method_name
    )

visualize_multiple_cams(
    image,
    heatmaps
)

if __name__ == "__main__":
    main()

```

Run:

```
python3 -m src.vision.compare_cam_methods
```

Analysis Questions

Answer:

- Do all CAM methods highlight the same image regions?
- Which method produces the sharpest visualization?
- Which method produces the smoothest visualization?
- Which method appears easiest to interpret?
- Do any methods produce suspicious or unexpected attention?
- What does this suggest about relying on only one XAI method?

Save conclusions:

```
reports/cam_methods_comparison.txt
```

18 Additional XAI Method: Occlusion Sensitivity Analysis

So far, you used Grad-CAM to visualize where the model focused its attention.

Grad-CAM answers:

Where does the model appear to look?

However, another important question is:

What happens if important information disappears?

To investigate this, you will implement:

Occlusion Sensitivity Analysis

The idea is simple:

- cover a small image region,
- run prediction again,
- measure confidence change,
- repeat across the image,
- build an importance map.

If hiding a region causes a large confidence drop:

That region was important for the model decision.

You will compare:

- Grad-CAM explanation,
- Occlusion explanation.

Pipeline

```
image
predict
cover small region
predict again
measure confidence difference
repeat over image
build sensitivity map
```

19 Task: Create Occlusion Script

Create:

src/vision/create_occlusion.py

Add imports:

```

from pathlib import Path

import torch
from torch import nn
from torchvision import models, transforms

from PIL import Image
import matplotlib.pyplot as plt
import numpy as np

```

Reuse:

```

MODEL_PATH
CLASS_NAMES_PATH
load_model()
load_class_names()
load_image()
predict()

```

from:

```
src/vision/create_gradcam.py
```

Import them:

```
from src.vision.create_gradcam import *
```

20 Task: Create Occlusion Function

Add:

```

def create_occlusion_map(
    model,
    image_tensor,
    predicted_class,
    patch_size=32,
    stride=16
):
    _, _, H, W = image_tensor.shape

    sensitivity = np.zeros(
        (H,W)
    )

    with torch.no_grad():
        original_output = model(
            image_tensor
        )

        original_probability = (
            torch.softmax(
                original_output,
                dim=1
            )[0,predicted_class]

```

```

        .item()
    )

    for y in range(
        0,
        H_patch_size,
        stride
    ):

        for x in range(
            0,
            W_patch_size,
            stride
        ):

            occluded = (
                image_tensor.clone()
            )

            occluded[
                :,
                :,
                y:y+patch_size,
                x:x+patch_size
            ] = 0

            with torch.no_grad():

                output = model(
                    occluded
                )

                probability = (
                    torch.softmax(
                        output,
                        dim=1
                    )[0,predicted_class]
                    .item()
                )

                drop = (
                    original_probability
                    -
                    probability
                )

                sensitivity[
                    y:y+patch_size,
                    x:x+patch_size
                ] += drop

            sensitivity -= (
                sensitivity.min()
            )

            sensitivity /= (
                sensitivity.max()
                +1e-8

```

```
)  
  
return sensitivity
```

21 Task: Visualize Results

Add:

```
def visualize_occlusion(  
    image,  
    sensitivity  
):  
  
    image = image.resize(  
        (224,224)  
    )  
  
    plt.figure(  
        figsize=(15,5)  
    )  
  
    plt.subplot(  
        1,2,1  
    )  
  
    plt.imshow(  
        image  
    )  
  
    plt.title(  
        "Original"  
    )  
  
    plt.axis(  
        "off"  
    )  
  
    plt.subplot(  
        1,2,2  
    )  
  
    plt.imshow(  
        sensitivity,  
        cmap="jet"  
    )  
  
    plt.title(  
        "Occlusion Sensitivity"  
    )  
  
    plt.axis(  
        "off"  
    )  
  
    plt.tight_layout()  
  
    plt.savefig(  
        "Occlusion Sensitivity.jpg"
```

```
"reports/gradcam_examples/occlusion.png"
)

plt.show()
```

22 Task: Main Function

Add:

```
def main():

    class_names = (
        load_class_names()
    )

    model = (
        load_model(
            class_names
        )
    )

    image_path = (
        "data/processed/images/"
        "test/river/river_0000.jpg"
    )

    image, tensor = (
        load_image(
            image_path
        )
    )

    predicted_class = (
        predict(
            model,
            tensor,
            class_names
        )
    )

    sensitivity = (
        create_occlusion_map(
            model,
            tensor,
            predicted_class
        )
    )

    visualize_occlusion(
        image,
        sensitivity
    )

if __name__=="__main__":
    main()
```

Run:

```
python3 -m src.vision.create_occlusion
```

23 Analysis Questions

Compare:

- Grad-CAM heatmap,
- Occlusion sensitivity map.

Answer:

- Do both methods highlight similar regions?
- Which explanation appears more localized?
- Which explanation appears easier to interpret?
- Did both methods identify the river region?
- What happens for unknown inputs?
- Can one explanation method completely replace another?

Save conclusions:

```
reports/occlusion_analysis.txt
```

Engineering Discussion

Grad-CAM and Occlusion answer slightly different questions:

- Grad-CAM:
Where does the model appear to focus?
- Occlusion:
What information is truly necessary for the prediction?

In real AI systems, engineers often use several explanation methods simultaneously. Different XAI methods may reveal different aspects of model behavior.

Independent Task 1: Compare Attention Across Multiple Images

Select at least:

- three forest images,
- three river images,
- three residential images.

Generate Grad-CAM visualizations for all images.

Questions:

- Does the model consistently focus on similar regions?
- Does attention vary significantly?
- Which class produces the most stable attention patterns?
- Which class appears more difficult?

Save:

```
reports/class_attention_consistency.txt
```

Independent Task 2: Investigate Sensitivity to Image Transformations

Real Earth Observation images may contain:

- different orientations,
- noise,
- illumination changes,
- sensor artifacts.

Modify the input image before prediction.

Examples:

- horizontal flip,
- rotation by 90 degrees,
- Gaussian blur,
- brightness modification,
- additional noise.

For each transformed image:

- run prediction,
- generate Grad-CAM,
- compare with original results.

Questions:

- Did confidence change?
- Did attention regions move?
- Which transformations had the largest impact?
- Which transformations had minimal impact?
- Does the model appear robust?

Save conclusions:

```
reports/transformation_analysis.txt
```

Independent Task 3: Design Your Own XAI Experiment

Design an independent experiment that investigates model behavior.

Possible ideas:

- compare multiple unknown EuroSAT classes,
- test additional noise levels,
- investigate image cropping,
- compare several images from the same class,
- investigate attention consistency,
- test Grad-CAM on deliberately modified images.

Requirements:

- clearly describe the goal,
- describe methodology,
- present observations,
- provide conclusions.

Save:

```
reports/custom_xai_experiment.txt
```

24 Submission

Submit:

- source code:

```
src/vision/create_gradcam.py
src/vision/create_occlusion.py
src/vision/compare_cam_methods.py
```

- generated visualizations:

```
reports/gradcam_examples/
```

- Grad-CAM analysis:

```
reports/gradcam_analysis.txt
```

- CAM methods comparison:

```
reports/cam_methods_comparison.txt
```

- Occlusion analysis:

```
reports/occlusion_analysis.txt
```

- independent task reports:

```
reports/class_attention_consistency.txt  
reports/transformation_analysis.txt  
reports/custom_xai_experiment.txt
```